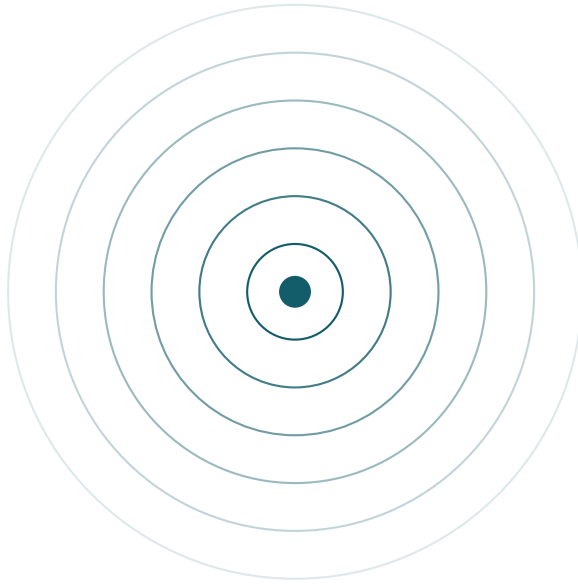


From one tap to the whole round trip



Web Applications from Zero

A Short Guide

FIRST EDITION

Web Applications from Zero

*What Happens Between Tapping a Button
and Seeing the Result*

FIRST EDITION

Web Applications from Zero: A Short Guide

Copyright © 2026 The Author. All rights reserved.

Revision History for the First Edition

2026-09-26 First release

The programs in this book are included for their instructional value. They have been tested with Python 3.14, FastAPI 0.135, and current versions of Chrome, Edge, and Firefox, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here change over time; check their current documentation.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with XeLaTeX. Source: Markdown, converted with pandoc.

Contents

Preface	v
Who This Book Is For	v
How This Book Is Organized	v
How to Use This Book	v
Conventions Used in This Book	vi
 Part I. Understand	 1
 1. Two Sides of Every App	 2
One App, Two Places	2
The Restaurant Picture	2
Client and Server	3
The Request and Response Loop	3
Why Split an App at All?	4
Try It	4
Summary, Key Terms, and Review Questions	4
 2. The Front End: What Runs on Your Screen	 6
The Server Sends Instructions, Not a Picture	6
Three Languages, Three Jobs	7
One Page, Every Screen	8
Front-End Frameworks	8
Mobile Apps Are Front Ends Too	8
What the Front End Can't Do	8
Try It	9
Summary, Key Terms, and Review Questions	9
 3. The Back End: What Runs Where You Can't See	 11
A Computer That Never Sleeps	11
Back-End Frameworks	11
Business Logic: The House Rules	12
The Database: Where Your Data Lives	13
How the Server Remembers You	13

APIs: The Contract Between the Two Sides	13
Try It	14
Summary, Key Terms, and Review Questions	14
4. Following One Tap from Start to Finish	16
The Button, Now for Real	16
The Whole Trip	17
Common Stacks	18
Try It	18
Summary, Key Terms, and Review Questions	19
Part II. What Next	20
5. Security, Speed, Failure, and Cost	21
Security: What Can Go Wrong	21
Speed: What Makes an App Feel Slow	22
Failure: What Breaks, and How to Tell Where	22
Cost: What It Costs to Run	22
6. Where to Go from Here	23
What to Learn Next	23
What to Skip for Now	23
Appendices	24
A. Glossary	24
Index	26

Preface

I use apps every day, but I have no idea what happens after I tap a button.

If that sounds like you, this short book is for you. It answers one question: *what happens between tapping a button and seeing the result?*

Who This Book Is For

You use apps and websites every day, but you have never built one. You may know basic Python, which helps with one or two short examples, but you don't need to know anything about the web, servers, or networks. This book explains; it doesn't teach you to build. When you finish, you will have a clear picture of how any app works, ready for the books that teach the building.

How This Book Is Organized

- Part I, *Understand* (Chapters 1–4), builds the picture. Chapter 1 shows the two sides of every app. Chapter 2 looks at the side on your screen, Chapter 3 at the side you can't see, and Chapter 4 follows one tap all the way through both.
- Part II, *What Next* (Chapters 5–6), asks what can go wrong, what makes an app slow, how it breaks, and what it costs, then points to what to learn next.
- The Glossary defines every new term in a line or two.

How to Use This Book

Read it in order; it is short, and each chapter builds on the one before. Chapters 1 to 4 each end with a *Try It* section that you do in your own web browser on a computer. Nothing needs installing. Do them: seeing the requests fly past makes the ideas stick.

Conventions Used in This Book

- Bold type marks a new term the first time it is explained. Every such term is also in the Glossary.
- Constant width marks code, commands, and anything you would type exactly.
- Code blocks show short examples and their real output. Commands are shown without a prompt.
- Three kinds of callout appear in the text:



A suggestion that saves time or trouble.



A general remark, or something to be aware of.



A mistake that can cost you data, money, or security.

PART I

Understand



What to ignore for now: specific frameworks, how to build a front end or back end, and network protocols in detail.

Two Sides of Every App

This chapter gives you the one picture the rest of the book hangs on. Every app you use has two sides: one in your hand, and one far away. The two sides talk by passing messages. By the end of the chapter you will know what each side is called and how their conversation works.

The problem. I tap a button, and something happens far away.

The question. What is on the other end of my tap, and how do the two ends talk?

One App, Two Places

You open a food delivery app, pick a pizza, and tap *Order*. A few seconds later the screen says “Order placed.” In between, the restaurant received your order, a driver was found, and your card was charged. None of that happened on your phone. Your phone doesn’t know which drivers are free, and it can’t charge a card by itself.

So the app you “use” is really two programs. One runs on your phone. The other runs on the delivery company’s computers, often hundreds of kilometres away. Together they make one **web application**: a program split between your device and a computer on the internet, with the two halves talking over the network. Email, maps, banking, social media, and on-line shopping all work this way.

The Restaurant Picture

A restaurant has two areas. The *front of house* is the dining room: tables, menus, waiters. Everything there is designed for you. The *back of house* is the kitchen. You never go there, but that is where the food, the recipes, and the cooks are.

Apps are built the same way, and developers even use the same words:

- The **front end** is the part you see and touch: screens, buttons, text boxes. It runs on your device.
- The **back end** is the part you never see: the program that keeps the data, applies the rules, and does the real work. It runs on the company's computers.

A waiter carries orders to the kitchen and plates back to the table. In an app, messages sent over the internet do the carrying.



You will also see “frontend”, “client side”, “backend”, and “server side”. They mean the same two things.

Client and Server

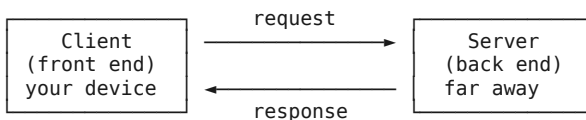
Two more words describe the *roles* in the conversation:

- The **client** is the program that asks. Your phone app is a client. So is a web browser.
- The **server** is the program that answers. It waits for questions, day and night, from many clients at once. People also say “server” for the computer it runs on.

One kitchen feeds every table, and one back end serves millions of phones. That is why shared data lives on the server: it is the one place every client can reach. Your friend sees your new photo because the photo is stored on the server, not on your phone.

The Request and Response Loop

Client and server talk in a strict pattern. The client sends a **request**: a message saying what it wants. The server sends back a **response**: a message with the answer. One request plus its response is a **round trip**.



Three rules hold for almost every round trip:

1. *The client speaks first.* The server answers questions; it doesn't start conversations.
2. *Every request gets one response*, even if the answer is “no” or “something broke”.
3. *Each round trip stands alone.* The server treats every request as a fresh message. (Chapter 3 shows how it still remembers who you are.)

Here is the pizza order as a round trip:

- Request: “Place an order: one margherita, to 12 Park Road, paid with the saved card.”
- Response: “Done. Order number 5531. The driver arrives in 25 minutes.”

Then the app shows the result on your screen. That is the whole shape of a web application: *tap, request, work, response, show*. The rest of this book fills in each step.

Why Split an App at All?

Why not put everything on the phone? Four reasons:

- *Sharing*. Data that many people need must live in one place everyone can reach.
- *Trust*. You control what runs on your own phone. A rule like “charge the card only after the restaurant accepts” must run where customers can’t tamper with it.
- *Power*. A server can search millions of restaurants in a blink. A phone has a small battery.
- *Updates*. Fix a bug on the server once, and every user gets the fix at the same moment.

The price of the split: the two sides must be able to reach each other. No network, no app. You have seen this price as “No internet connection.”

Try It

You can watch round trips happen. Your browser has **developer tools**: a built-in panel that shows what a web page is doing behind the scenes.

1. Open any news website in Chrome, Edge, or Firefox on a computer.
2. Open the developer tools: press F12 (Windows, Linux) or Cmd+Option+I (Mac), or right-click the page and choose *Inspect*.
3. Click the *Network* tab, then reload the page.

Expected result: The panel fills with a long list, often over a hundred lines. Each line is one round trip. The *Status* column shows numbers such as 200, which means “OK”, and the bar at the bottom counts the requests. One page, loaded by one click, needed all of them.

Summary, Key Terms, and Review Questions

Summary

- A web application is split in two: a front end on your device and a back end on the company’s computers.

- The client asks and the server answers. One server answers many clients.
- Client and server talk in round trips: one request, one response.
- Apps are split for sharing, trust, power, and easy updates. The cost is that both sides need the network.

Key Terms

Term	Meaning
Web application	A program split between your device and a server
Front end	The part you see and touch
Back end	The hidden part that keeps data and does the work
Client	The program that asks
Server	The program that answers
Request	The message the client sends
Response	The message the server sends back
Round trip	One request plus its response
Developer tools	The browser panel that shows what a page is doing

Review Questions

1. Using the restaurant picture, explain the front end and the back end.
2. Why does your friend's phone show a photo you uploaded from your phone?
3. Walk through “tap, request, work, response, show” for sending a chat message.
4. Give two reasons a rule like “charge the card” runs on the server, not the phone.

You understand this when you can pick any app on your phone and say which of its jobs happen on your device and which happen far away.

The Front End: What Runs on Your Screen

This chapter looks at the side of the app you can see: who draws the screen, the three languages every web page is made of, what a front-end framework adds, and why a phone app is a front end too.

The problem. The same app looks right on my laptop and my phone, but who draws it?

The question. How does an app become the picture on my screen, and how does that picture fit every size of screen?

The Server Sends Instructions, Not a Picture

When you open a website, the server does not send a finished picture of the page. It sends *instructions*, and your device draws the page itself.

Think of flat-pack furniture. The shop doesn't deliver a built bookcase. It sends the parts and an instruction sheet, and you assemble it at home. A good sheet even says: "If your wall is narrow, stack the shelves in one column."

The program that does the assembling is the **browser**: Chrome, Safari, Firefox, or Edge. It downloads the instructions, follows them, and draws the result. That is why the same app looks right on a laptop and a phone: each device builds the page to fit its own screen.

Three Languages, Three Jobs

The instructions come in three languages:

Language	Its job	In the furniture picture
HTML	What is on the page: text, buttons, images	The list of parts
CSS	How it looks: colours, fonts, layout	The paint and arrangement
JavaScript	What happens when you touch it	The drawers that slide

Here is a tiny but complete page that uses all three:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1 { font-family: sans-serif; }
    button { background: royalblue; color: white; }
  </style>
</head>
<body>
  <h1>My Notes</h1>
  <button id="save">Save</button>
  <p id="status"></p>
  <script>
    const button = document.getElementById("save");
    button.onclick = function () {
      document.getElementById("status").textContent = "Saved!";
    };
  </script>
</body>
</html>
```

The CSS inside `<style>` makes the heading plain and the button blue. The HTML inside `<body>` lists a heading, a button, and an empty paragraph. The JavaScript inside `<script>` says: when someone clicks the button, put *Saved!* in the empty paragraph. Open this file in a browser and click *Save*, and *Saved!* appears under the button.



This button is a fake. It changed the page, but nothing left your computer, so nothing was saved. A real Save button must send a request to a back end. Chapter 4 shows this same button doing exactly that.

One Page, Every Screen

A phone screen is narrow; a laptop screen is wide. Designers don't build two websites. They write CSS rules that change the layout to fit the space: "on a wide screen, show the menu on the left; on a narrow one, hide it behind a button." This is **responsive design**: the "stack the shelves if the wall is narrow" line of the instruction sheet.

Front-End Frameworks

The example page has one button. An online shop has thousands, and changing one number can mean five places on the screen must change with it. Doing that by hand in JavaScript is slow and error-prone.

A **front-end framework** is ready-made code that does this bookkeeping. React, Vue, Angular, and Svelte are well-known ones. The developer describes what the screen should look like for the current data ("the cart shows these three items"), and the framework updates the page whenever the data changes.

Frameworks build screens from **components**: small, reusable pieces such as a product card or a like button. Like building bricks, one component can appear hundreds of times on a page.



A framework helps the developer; it is not a new language for the browser. A React app still reaches your browser as HTML, CSS, and JavaScript, the only three languages a browser draws.

Mobile Apps Are Front Ends Too

Apps you install from an app store are usually **native apps**: programs written for one kind of phone, in languages like Swift (iPhone) or Kotlin (Android). They draw their own screens without a browser.

The tools differ, but the role is the same. A native app shows screens, reacts to taps, and asks a back end for anything it doesn't have. Often one back end serves the website, the iPhone app, and the Android app together.

What the Front End Can't Do

Everything in the front end travels to the user's device, where the user can read it and change it. So the front end can't keep secrets, and it can't make final decisions. It can say "this coupon looks valid", but only the back end can accept it.



Never put a password, a secret key, or a price the customer must not change in front-end code. The back end must check everything the front end sends.

Try It

See the instructions behind a real page, and change them.

1. Open any website on a computer. Right-click a heading and choose *Inspect*. The *Elements* tab shows the page's HTML, with your heading highlighted.
2. Double-click the heading's text in that panel, type something new, and press Enter. Then reload the page.
3. Click the phone-and-tablet icon at the top of the developer tools (the device toolbar) and pick a phone.

Expected result: In step 2 the heading changes to your text, then goes back when you reload. You changed only your own copy of the instructions; the server never knew. In step 3 the page rearranges itself for a narrow screen: responsive design at work.

Summary, Key Terms, and Review Questions

Summary

- The server sends instructions; the browser follows them and draws the page.
- HTML says what is on the page, CSS says how it looks, and JavaScript says what it does.
- Responsive design lets one page fit any screen.
- Front-end frameworks keep large screens up to date, built from reusable components.
- Native apps are front ends too, often sharing one back end with the website.
- The front end can't keep secrets or make final decisions.

Key Terms

Term	Meaning
Browser	The program that downloads and draws web pages
HTML	The language for what is on a page
CSS	The language for how a page looks
JavaScript	The language for what a page does
Responsive design	One layout that adapts to any screen size
Front-end framework	Ready-made code that keeps a screen in step with its data
Component	A small, reusable piece of a screen
Native app	An app written for one kind of phone

Review Questions

1. Why can the same website look right on a laptop and a phone?
2. For a *Like* button, what does the HTML do, what does the CSS do, and what does the JavaScript do?
3. What problem does a front-end framework solve?
4. In what way is an iPhone app the same thing as a website?
5. Why can't a shop let the front end decide the final price?

You understand this when you can look at any screen in any app and say what the front end drew by itself and what it must have asked the back end for.

The Back End: What Runs Where You Can't See

This chapter walks into the kitchen: what the back end does with each request, where your data ends up, how the server remembers you, and how the two sides agree on what they may ask each other.

The problem. Where does my data go when I press Save?

The question. What happens on the far side of the app, and how does it keep my data and remember me?

A Computer That Never Sleeps

A back end runs on a server: usually a computer with no screen in a *data centre*, a warehouse full of such computers. Most companies rent these machines from a cloud company.

The back-end program *listens*: it waits for a request, answers it, and waits again. For each request it does three jobs, like a kitchen handling an order slip:

1. *Routing*: send the order to the right station. Which piece of code handles “save a note”?
2. *Business logic*: follow the recipe and the house rules. Is this allowed? Does it make sense?
3. *Storing and fetching*: get ingredients from the store room. Read or write the database.

Then it builds a response and sends it back.

Back-End Frameworks

Every back end needs the same plumbing: receive requests, find the right code, send responses. A **back-end framework** provides it, so developers write only what is special to their app. Here is a complete, tiny back end written with FastAPI, a Python framework:

```

from fastapi import FastAPI

app = FastAPI()
notes = [] # the app's memory, for now

@app.post("/notes")
def save_note(note: dict):
    notes.append(note["text"])
    return {"saved": True, "count": len(notes)}

@app.get("/notes")
def list_notes():
    return {"notes": notes}

```

The two functions are ordinary Python. The line above each, starting with @, is a **route**: it tells the framework, “when a request to save a note (POST /notes) arrives, run this function.” A function that answers a request is a **handler**. Whatever it returns becomes the response.

You don’t need to run this; it is here to show how small a handler can be. Saved as `app.py` and started with `uvicorn app:app` (a program that runs FastAPI apps), it listens on your own computer, called `localhost`. `curl`, a terminal program that sends one request and prints the response, can then play the client:

```

curl -X POST http://localhost:8000/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "Buy milk"}'

{"saved":true,"count":1}

```

Business Logic: The House Rules

The **business logic** is the set of rules that makes an app *this* app:

- A cinema can’t sell the same seat twice.
- A bank can’t let you send more money than you have.
- Only the person who wrote a post can edit it.
- The shop, not your browser, adds up the price of your cart.

These rules live on the back end because it is the one place the company fully controls.

The Database: Where Your Data Lives

Our tiny back end keeps notes in a Python list. Stop the server, start it again, and ask for the list:

```
curl http://localhost:8000/notes
```

```
{"notes": []}
```

The milk is gone: a list in memory vanishes when the program stops. Real apps use a **database**: a program built to keep data safely on disk, find any piece of it quickly, and let many users read and write at once. Picture a huge filing room with a fast, careful clerk. PostgreSQL, MySQL, and MongoDB are well-known ones.

So here is the answer to this chapter's problem. When you press Save, your data travels to the back end, which checks it against the rules and hands it to the database. Your phone may keep a copy for speed, but the real copy lives there.

How the Server Remembers You

The server greets every request like a stranger. So how does it know this request comes from *you*, who logged in ten minutes ago?

Think of a cloakroom. You hand over your coat and get a numbered ticket. Later you show the ticket, not your face, and get your coat back.

When you log in, the back end checks your password and writes down a **session**: a record saying “ticket 7f3a9c belongs to Priya, logged in at 9:14.” It gives the ticket to your browser as a **cookie**: a small piece of text the browser stores and sends back with every later request to that site. Mobile apps use a similar ticket, usually called a token.



Whoever holds your ticket *is* you, as far as the server knows. Never paste cookie values anywhere, and log out on shared computers: that tears up the ticket.

APIs: The Contract Between the Two Sides

The front end and the back end are often built by different people. They need an agreement: which requests the back end accepts, what each must contain, and what comes back. That agreement is the **API** (Application Programming Interface). Each thing it offers, such as “save a note” at POST /notes, is an **endpoint**. The data usually travels as **JSON**: plain text that looks almost exactly like a Python dictionary, such as {"text": "Buy milk"}.

The API is the restaurant's menu. As long as the menu stays the same, the kitchen can buy new stoves and the dining room can be repainted without breaking each other. And a web-site, an iPhone app, and an Android app can share one back end: they all order from the same menu.

Try It

Look at the cloakroom tickets your browser holds for a site.

1. Open a private (incognito) window and log in to a site you use, such as web email.
2. Open the developer tools. In Chrome or Edge, go to *Application*, then *Cookies*; in Firefox, *Storage*, then *Cookies*. Click the site's name.
3. Right-click the site's name, choose *Clear* (Chrome, Edge) or *Delete All* (Firefox), and reload the page.

Expected result: In step 2 you see several cookies, each a name and a value; the login one usually has a long, random value. In step 3 you are suddenly logged out: you threw away your tickets, so the server no longer knows you.

Summary, Key Terms, and Review Questions

Summary

- A back end listens for requests on a server. For each one it routes, applies business logic, and reads or writes the database.
- Back-end frameworks provide the plumbing; developers write handlers.
- Data in memory vanishes; a database keeps it safe and shared.
- A session and a cookie let the server know who you are on every request.
- The API is the contract between the two sides, usually spoken in JSON.

Key Terms

Term	Meaning
Back-end framework	Ready-made plumbing for answering requests
Route	A rule linking a kind of request to its handler
Handler	A function that answers one kind of request
Business logic	The app's own rules
Database	A program that stores data safely and shares it
Session	The server's record of who is logged in
Cookie	A small piece of text the browser keeps and sends back
API	The agreement on what the front end may ask the back end
Endpoint	One thing an API offers, such as <code>POST /notes</code>
JSON	A plain-text data format, like a Python dictionary

Review Questions

1. What three jobs does a back end do for every request?
2. In the tiny FastAPI app, what is the route and what is the handler for saving a note?
3. Why did the note disappear when the server restarted, and what fixes that?
4. Explain sessions and cookies using the cloakroom picture.
5. How can a company rebuild its back end without breaking the apps that use it?

You understand this when you can answer “where does my data go when I press Save?” for any app, naming the handler, the rules it checks, and the database, and explain how the server knows it was you.

Following One Tap from Start to Finish

This chapter puts the two sides together. You will follow one tap on a Save button to the server and back, then meet the common combinations of tools real apps are built from.

The problem. I know the two sides now, but not how one tap travels between them.

The question. What happens, in order, between tapping Save and seeing “Saved!”?

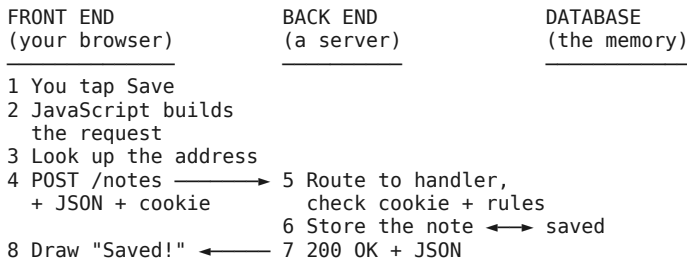
The Button, Now for Real

In Chapter 2, the Save button only changed the page. Here is the same button’s JavaScript, now talking to the back end from Chapter 3:

```
button.onclick = async function () {  
  const response = await fetch("/notes", {  
    method: "POST",  
    headers: {"Content-Type": "application/json"},  
    body: JSON.stringify({text: "Buy milk"}),  
  });  
  const data = await response.json();  
  document.getElementById("status").textContent =  
    "Saved! Notes so far: " + data.count;  
};
```

fetch sends a request; await means “wait here for the response.” The last lines put the result on the page: *Saved! Notes so far: 1*. Everything in this chapter happens inside that one await.

The Whole Trip



1. *The tap.* The browser notices the click and runs the button's JavaScript.
2. *The request.* JavaScript builds a request: an action (POST, meaning "create"), an address, and a JSON body. The browser attaches the site's cookie by itself.
3. *The address.* Every request goes to a **URL**, such as `https://notes.example.com/notes`. The middle part, `notes.example.com`, is a **domain name**: a human-friendly name for the server. Computers really find each other by numbers called **IP addresses**, so the browser asks **DNS**, the internet's phone book, to turn the name into a number. The `https` at the front means **HTTPS**: the request travels sealed, so nobody on the way can read it.
4. *The journey.* The sealed request hops across the internet to the server.
5. *The back end.* The framework matches `POST /notes` to its route and runs the handler, which looks up the cookie to learn who you are and applies the rules.
6. *The database.* The handler asks the database to store the note and waits for "OK".
7. *The response.* The back end sends back a **status code**, a number that says how it went, plus JSON. Here is that response exactly as it arrives, shown by `curl -i` (the `-i` means "include the headers"):

```
curl -i -X POST http://localhost:8000/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "Buy milk"}'

HTTP/1.1 200 OK
date: Sat, 26 Sep 2026 14:31:05 GMT
server: uvicorn
content-length: 24
content-type: application/json

{"saved":true,"count":1}
```

The first line is the status. Then come headers, notes about the message. After the blank line comes the body: the handler's JSON.

8. *The drawing.* JavaScript reads the JSON and changes one paragraph. The browser redraws only that part; the page doesn't reload.

The whole trip usually takes a tenth to a third of a second. You feel it as “instant”.



Status codes come in families. Numbers starting with 2 mean success, with 4 mean “your request was wrong” (404 is “not found”), and with 5 mean “the server broke”. The first digit tells you which side to blame.

Common Stacks

Every app picks a front end, a back end, and a database. That combination is its **stack**. The pieces mix freely, because they only talk through requests and responses:

Stack	Front end	Back end	Database
Python	React or plain HTML	FastAPI or Django	PostgreSQL
JavaScript everywhere	React	Node.js with Express	MongoDB
Classic “LAMP”	HTML made by PHP	PHP on Linux	MySQL
Ruby on Rails	Rails pages	Ruby on Rails	PostgreSQL
Mobile	Swift or Kotlin app	Any of the above	Any

A developer who works on both sides is called a **full-stack developer**. The tools change every few years; the trip you just followed does not.

Try It

Catch a real request in the act.

1. Open a site with a search box that suggests results as you type, such as an online shop.
2. Open the developer tools, click *Network*, and choose the *Fetch/XHR* filter. This hides images and files and shows only requests made by JavaScript. Clear the list with the clear button.
3. Type one letter into the search box. Click the new request that appears.

Expected result: A request appears as you type, with status 200. *Headers* shows its URL and method, *Response* (or *Preview*) shows JSON holding the suggestions, and *Timing* shows how long the server took. Match each piece to the eight steps above.

Summary, Key Terms, and Review Questions

Summary

- A tap runs JavaScript, which builds a request and sends it to a URL.
- DNS turns the domain name into an IP address; HTTPS seals the request for the journey.
- The back end routes the request, checks who you are and the rules, and uses the database.
- The response carries a status code and JSON; JavaScript redraws part of the page.
- A stack is one app's choice of front end, back end, and database.

Key Terms

Term	Meaning
URL	The full address a request is sent to
Domain name	A human-friendly name for a server
IP address	The number computers use to find each other
DNS	The internet's phone book, from names to numbers
HTTPS	The sealed, encrypted way requests travel
Status code	A number saying how a request went
Stack	An app's front end, back end, and database
Full-stack developer	Someone who works on both sides

Review Questions

1. List the eight steps between tapping Save and seeing "Saved!".
2. What does DNS do, and why is it needed?
3. What are the three parts of the response shown by `curl -i`?
4. What would a status code of 500 tell you, and what would 404 tell you?
5. Why can a React front end work with a Django back end?

You understand this when you can draw the whole trip from memory for any button in any app, and point to the step where each piece of this book fits.

PART II

What Next

Security, Speed, Failure, and Cost

Every web application, from a to-do list to a bank, faces the same four questions: what can go wrong, what makes it slow, how it breaks, and what it costs. This chapter answers each one using the picture you now have of the two sides and the trip between them.

The problem. My app works on my screen. But could someone break in, could it feel slow, could it fall over, and who pays for it?

The question. What can go wrong with a web application, and what keeps each kind of trouble small?

Security: What Can Go Wrong

The front end lives on the user's device, so the user controls it. Anyone can skip your page and send requests with a tool like `curl`. So the back end must treat every request as a stranger at the door and check three things: *who are you* (the session), *are you allowed* (the rules), and *does this make sense* (a negative price? a 10 MB name?).

The other dangers follow the trip:

- *On the device:* secrets in front-end code are not hidden. Keep them on the back end.
- *On the way:* a stolen cookie is a stolen login. HTTPS seals every request, and sessions that expire limit how long a stolen ticket works.
- *In the database:* a leak exposes everything stored. Keep only what you need, and never store passwords as plain text. Store them **hashed**: scrambled one way, so that even a leak doesn't reveal them.

What limits the damage: give each part only the access it needs, so one mistake can't open every door.

Speed: What Makes an App Feel Slow

Time goes to three places on the trip:

- *Distance*. Every round trip takes time; twenty trips take twenty times as long.
- *Size*. Big images and heavy JavaScript download slowly, especially on a phone.
- *Work*. A back end that searches a huge database carelessly can take seconds.

The fixes match. A **cache** keeps a copy of an answer close by, so the next request travels less far, or not at all. A **CDN** (content delivery network) keeps copies of files on servers near the users. Shrink images and code. And *feel* faster: show the new note at once, while the request is still travelling.

Failure: What Breaks, and How to Tell Where

When something goes wrong, first find which step of the trip failed. The developer tools tell you most of it:

What you see	Where it likely failed	Where to look
A button does nothing	Front end: a JavaScript error	The <i>Console</i> tab
“Can’t reach this site”	The network or DNS	Try another site
An endless spinner	Back end slow or down	<i>Network</i> tab
A status starting with 4	Your request (401, 404)	<i>Network</i> tab
A status starting with 5	The back end broke	The server’s logs

A client gives up after a **timeout**, a set time such as thirty seconds. On the back end, developers read **logs**: lines the server writes about every request, such as "POST /notes HTTP/1.1" 200 OK from Chapter 3’s back end.

Cost: What It Costs to Run

The two sides cost very different amounts:

- *The front end* is cheap to serve: its files are sent once and cached, and the drawing happens on the user’s device.
- *The back end* runs day and night, and that is where the bills are: servers, the database (often the biggest item), data sent to users, and paid services such as email.

Costs grow with users, though free tiers let a small app run almost free. The largest cost is usually people, who build, update, and protect both sides.

Where to Go from Here

You now know what happens between tapping a button and seeing the result. Here is what to learn next, and what to leave alone for now.

What to Learn Next

Each step of the trip has its own book in this series:

- The journey between the two sides: *Networking from Zero*.
- The contract and the back end: *APIs to FastAPI*.
- Where the data lives: *Databases from Zero*.
- The tools every developer uses: *Linux and Git from Zero*.
- Getting a back end onto real servers: *Infrastructure Engineering from Zero*.
- Putting it all together: *The Complete Software Engineering Guide*.

For the front end, learn plain HTML, CSS, and JavaScript first, and pick a framework only after that. MDN Web Docs is a free, reliable reference for all three.

What to Skip for Now

- *Choosing the “best” framework*. They all do the jobs in Chapters 2 and 3. Learn one well.
- *Scaling to millions of users*. Microservices and Kubernetes solve problems you don’t have yet.
- *Other ways to talk*. WebSockets, GraphQL, and server-side rendering are variations on the same trip. Meet them when a project needs them.

Whatever you learn next, keep the trip in mind: tap, request, work, response, show. Every new tool is a better way to do one of those steps.

Glossary

Term	Meaning
API	Application Programming Interface: the agreement on what a back end can be asked and what it answers
Back end	The hidden part of an app that keeps the data and does the work
Back-end framework	Ready-made plumbing for answering requests (FastAPI, Django, Express)
Browser	The program that downloads web pages and draws them (Chrome, Safari, Firefox)
Business logic	The app's own rules, such as "never sell the same seat twice"
Cache	A nearby copy of an answer, kept so it doesn't have to be fetched again
CDN	Content delivery network: servers worldwide keeping copies of files near users
Client	The program that asks: a browser, a phone app, or a tool like <code>curl</code>
Component	A small, reusable piece of a screen, such as a like button
Cookie	A small piece of text the browser keeps and sends back with every request to a site
CSS	The language that says how a page looks: colours, fonts, and layout
Database	A program that keeps data safe on disk, finds it fast, and shares it
Developer tools	The panel built into a browser that shows what a page is doing
DNS	Domain Name System: the internet's phone book, turning domain names into IP addresses
Domain name	A human-friendly name for a server, such as <code>notes.example.com</code>
Endpoint	One thing an API offers, such as "save a note" at <code>POST /notes</code>
Front end	The part of an app you see and touch, running on your device
Front-end framework	Ready-made code that keeps a screen in step with its data (React, Vue)
Full-stack developer	A developer who works on both the front end and the back end
Handler	A back-end function that answers one kind of request

Term	Meaning
Hashed	Scrambled one way, as passwords are stored, so the original can't be read back
HTML	The language that says what is on a page: text, buttons, images
HTTPS	The sealed, encrypted way requests and responses travel
IP address	The number computers use to find each other on the internet
JavaScript	The language that says what a page does when you touch it
JSON	A plain-text data format that looks like a Python dictionary
Logs	Lines a server writes about each request
Native app	An app written for one kind of phone, such as iPhone or Android
Request	The message a client sends to ask for something
Response	The message a server sends back with the answer
Responsive design	CSS rules that let one page fit any screen size
Round trip	One request plus its response
Route	A rule linking a kind of request, such as <code>POST /notes</code> , to its handler
Server	The program that answers requests, or the computer it runs on
Session	The server's record of who is logged in, looked up by a cookie
Stack	The front end, back end, and database an app is built from
Status code	A number saying how a request went (200 OK, 404 Not Found)
Timeout	The time a client waits for a response before giving up
URL	The full address a request is sent to
Web application	A program split between your device and a server

Index

A

API, 13

B

back end, 3

back-end framework, 11

browser, 6

business logic, 12

C

cache, 22

CDN, 22

client, 3

components, 8

cookie, 13

CSS, 7

D

database, 13

developer tools, 4

DNS, 17

domain name, 17

E

endpoint, 13

F

front end, 3

front-end framework, 8

full-stack developer, 18

H

handler, 12

hashed, 21

HTML, 7

HTTPS, 17

I

IP addresses, 17

J

JavaScript, 7

JSON, 13

L

logs, 22

N

native apps, 8

R

request, 3

response, 3

responsive design, 8

round trip, 3

route, 12

S

server, 3

session, 13

stack, 18

status code, 17

T

timeout, 22

U

URL, 17

W

web application, 2

Web Applications from Zero

You use apps every day, but what actually happens after you tap a button? This short book answers that one question, from zero. It shows the two sides of every app, the front end on your screen and the back end you never see, and follows a single tap all the way to the server and back.

You'll learn:

- What the front end and the back end each do
- How the server keeps your data and remembers who you are
- What happens, step by step, between a tap and the result
- What can go wrong, what makes apps slow, and what they cost